

Distributed Ticket Booking System

Milestone 1 Progress Report

Course: CS G623 — Advanced Operating Systems

Institution: BITS Pilani

Architecture: Option A (Distributed Ticket Booking System)

Milestones: 1 of 2 (Milestone 1 Complete)

The Core Problem

- **High Contention:** Multiple clients competing simultaneously for scarce resources.
- **Race Conditions:** Naive read-then-decrement leads to overbooking and inconsistent state.
- **AOS Objective:** Enforce mutual exclusion, fairness, and inter-process coordination using gRPC without external databases.

Domain-Agnostic Methodology

The core distributed engine is completely generic and adapts to any scarce resource:

- **Vaccine Appointments:** VaccineSlot (dose, hospital, date)
- **Concert / Cinema:** Seat reservations per row / section
- **Aviation:** Seat allocation per flight leg
- **Cloud Infrastructure:** VM / GPU allocation across compute clusters

Only domain attributes and validation rules change; locking, queues, and gRPC remain identical.

Distributed Architecture

Three independent nodes communicating strictly over **gRPC**:

- **Node 5 (Clients)**: Concurrent client processes performing login, queries, and bookings.
- **Node 2 (App Server - :50051)**: In-memory inventory, per-slot lock mutexes, session management, mock payment.
- **Node 1 (LLM Server - :50052)**: Offline FAQ chatbot with dynamic query batching.

System Architecture Diagram



Core RPC Contracts

Strictly adheres to assessment function signatures:

- `login(user, pass)` → `LoginResponse(status, token)`
- `logout(token)` → `StatusResponse(status)`
- `get(token, type, params)` → `GetResponse(status, items)`
- `post(token, type, data)` → `StatusResponse(status)`
- `getLLMAnswer(reqId, query, ctx)` → `GetLLMAnswerResponse(reqId, ans)`

Concurrency: Per-Slot Locks

- **The Global Lock Flaw:** A single mutex serializes all requests, creating an artificial throughput bottleneck.
- **Our Solution:** Dedicated `threading.Lock` per inventory item.
- **Parallel Execution:** Clients booking *different* slots proceed simultaneously with zero lock contention.
- **Mutual Exclusion:** Only requests targeting the *identical* slot serialize.

Request Dispatcher & Lock Manager

Every mutation passes through two coordination components:

1. **Dispatcher (FIFO Queue):** Orders incoming booking requests; supports priority tagging (e.g., emergency / VIP).
2. **LockManager:** Tracks logical lock ownership and waiting queues for distributed coordination.
3. **Execution:** Requests acquire the slot lock, mutate state, release the lock, and dequeue cleanly.

Session & State Management

- **No External Database:** All state resides in memory per Milestone 1 requirements.
- **Token Authentication:** Secure UUID-based session tokens issued on login, verified on each call, deleted on logout.
- **In-Memory Store:** Thread-safe slot inventories and confirmed appointment maps.
- **Clean Reset:** State is transient for Milestone 1; replicated persistence follows in Milestone 2.

Offline Local LLM (Node 1)

- **Model:** HuggingFace `Qwen/Qwen2.5-0.5B-Instruct` running entirely on Node 1.
- **100% Offline:** Model loads from local cache (`local_files_only=True`). Zero external API dependency or data leakage.
- **Hardware-Dependent Speed:**
 - CPU: 3–8s per query
 - GPU / Apple Silicon (MPS): < 1 s per query

Dynamic Query Batching

- **Problem:** Sequential neural inference incurs repeated fixed latency per request.
- **Batching Window (60 ms):** Incoming gRPC queries within 60 ms are gathered into a single pipeline batch.
- **Parallel Evaluation:** The model runs one unified forward pass for all queued queries.
- **Throughput:** Near $N \times$ throughput improvement under simultaneous client arrivals.

Live Context Injection

- **Real-time Awareness:** When users query slot availability via the LLM, Node 2 detects availability keywords.
- **Context Bridge:** Node 2 queries `BookingService` for live counts and injects them into the LLM prompt.
- **Accurate Answers:** The LLM responds with live inventory numbers rather than vague static instructions.

Verification & Test Results

- **Unit & Contract Suite:** 13/13 tests passing across models, dispatcher, lock manager, and service contracts.
- **Concurrency Stress Test:**
 - 10 concurrent client threads racing for 3 remaining slots.
 - **3 Accepted, 7 Rejected, 0 Overbookings.**
 - Confirmed 100% mutual exclusion and inventory consistency.

Milestone 2 Roadmap

- **Raft Consensus Protocol:** Distributed leader election among App Server replicas.
- **Log Replication:** State replicated across a quorum before booking commit.
- **Fault Tolerance:** Automatic failover and zero data loss on leader crash.
- **Persistent Consensus:** Transition from in-memory state to durable distributed logs.